my-agents-cli

`google-agents-cli` (`1.13.0`) 스캐폴딩 명령어로 생성된 표준 엔터프라이즈급 ReAct 에이전트 프로젝트입니다.
FastAPI 기반 백엔드, A2A(Agent-to-Agent) 통신, Vertex AI Reasoning Engine 어댑터, 자동화된 단위/통합/평가 테스트, 그리고 Terraform 인프라 배포 코드까지 포함된 종합 프로젝트 구성을 갖추고 있습니다.

---

1. 프로젝트 상세 폴더 구조 및 구성 요소 분석

```
my-agents-cli/
├── 📁 app/                          # [초기 생성] 핵심 에이전트 애플리케이션 코드
│   ├── 📄 agent.py                  # 🟢 [초기 생성] 메인 에이전트 로직 및 Mock 도구(날씨, 현재시각) 정의
│   ├── 📄 fast_api_app.py           # 🟢 [초기 생성] REST / SSE / A2A 지원 FastAPI 백엔드 서버
│   ├── 📄 __init__.py               # 🟢 [초기 생성] Python 패키지 초기화 파일
│   ├── 📁 app_utils/                # 🟢 [초기 생성] 에이전트 부가 유틸리티 모듈
│   │   ├── 📄 a2a.py                # 🟢 [초기 생성] A2A 프로토콜 통신 및 Agent Card 처리
│   │   ├── 📄 reasoning_engine_adapter.py # 🟢 [초기 생성] Vertex AI Reasoning Engine 라우트 어댑터
│   │   └── 📄 services.py           # 🟢 [초기 생성] ADK 세션 및 아티팩트 서비스 팩토리
│   └── 📁 .adk/                     # ⚡ [런타임 동적 생성] 로컬 실행 시 자동 생성되는 내부 데이터 폴더
│       └── 📄 session.db            # ⚡ [런타임 동적 생성] 로컬 세션 및 대화 상태 저장 SQLite DB
│
├── 📁 deployment/                   # ⚡ [scaffold enhance 생성] 인프라 및 배포 코드 디렉터리
│   └── 📁 terraform/                # ⚡ [동적 생성] GCP 리소스 배포용 Terraform HCL 코드
│       ├── 📁 shared/               # ⚡ [동적 생성] BigQuery 로그 스키마 등 공유 인프라 모듈
│       └── 📁 single-project/       # ⚡ [동적 생성] 단일 GCP 프로젝트 배포 스크립트 (service.tf,
iam.tf 등)
│
├── 📁 tests/                        # 🟢 [초기 생성] 테스트 및 평가 파이프라인
│   ├── 📁 unit/                     # 🟢 [초기 생성] 단위 테스트 디렉터리
│   │   └── 📄 test_dummy.py         # 🟢 [초기 생성] 기본 더미 단위 테스트
│   ├── 📁 integration/              # 🟢 [초기 생성] 서버 통합 테스트 디렉터리
│   │   ├── 📄 test_agent.py         # 🟢 [초기 생성] 에이전트 로직 통합 테스트
│   │   └── 📄 test_server_e2e.py    # 🟢 [초기 생성] FastAPI 및 SSE 엔드포인트 E2E 테스트
│   └── 📁 eval/                     # 🟢 [초기 생성] 에이전트 동작 평가(Evaluation) 파이프라인
│       ├── 📄 eval_config.yaml      # 🟢 [초기 생성] 평가 지표 및 설정 명세
│       ├── 📄 response_quality.py   # 🟢 [초기 생성] LLM 응답 품질 자동 채점기
│       └── 📁 datasets/             # 🟢 [초기 생성] 평가 데이터셋 폴더
│           ├── 📄 basic-dataset.json # 🟢 [초기 생성] 평가 질의 및 기대 응답 데이터
│           └── 📄 README.md         # 🟢 [초기 생성] 데이터셋 안내
│
├── 📁 .venv/                        # ⚡ [동적 생성 - 패키지 설치] uv 기반 Python 가상환경
├── 📁 .google-agents-cli/           # ⚡ [런타임 동적 생성] 로컬 서버 실행 로그 디렉터리
│   ├── 📄 run_server.log            # ⚡ [동적 생성] 서버 표준 출력 로그
│   └── 📄 run_server.stderr.log     # ⚡ [동적 생성] 서버 에러 로그
│
├── 📄 agents-cli-manifest.yaml      # 🟢 [초기 생성] agents-cli 메타데이터 명세서
├── 📄 pyproject.toml                # 🟢 [초기 생성] uv 기반 프로젝트 의존성 및 패키지 설정
├── 📄 uv.lock                       # ⚡ [동적 생성 - 패키지 설치] 정확한 패키지 버전 잠금 파일
├── 📄 deployment_metadata.json      # ⚡ [동적 생성 - 배포] GCP 원격 엔진 ID 및 배포 이력
├── 📄 Dockerfile                    # 🟢 [초기 생성] 컨테이너 기반 빌드 및 배포 도커 명세서
├── 📄 .env                          # 🟢 [초기 생성 / 설정] 로컬 환경 변수 파일 (API 키 등)
├── 📄 .env.example                  # 🟢 [초기 생성] 환경 설정 템플릿
├── 📄 .gitignore                    # 🟢 [초기 생성] Git 제외 규칙
├── 📄 .gcloudignore                 # 🟢 [초기 생성] GCP 빌드 제외 규칙
├── 📄 GEMINI.md                     # 🟢 [초기 생성] AI 어시스턴트용 프로젝트 컨텍스트 가이드
├── 📄 README.md                     # 📄 [영문 안내 문서] 프로젝트 기본 영문 가이드
└── 📄 README-KR.md                  # 📄 [한글 안내 문서] 상세 한글 가이드 (본 파일)
```

2. 요구 사항 (Requirements)

시작하기 전에 다음 도구들이 설치되어 있는지 확인하세요:

- **uv**: Python 패키지 관리자 - 설치 안내 (`uv add <패키지>` 로 패키지 추가)
- **agents-cli**: Google Agents CLI - `uv tool install google-agents-cli` 로 설치
- **Google Cloud SDK**: GCP 서비스 이용을 위한 SDK - 설치 안내

---

3. 실행 방법 (How to Run)

`my-agents-cli` 디렉터리 내부에서 명령어를 실행합니다:

```
# 디렉터리 이동
cd my-agents-cli

# 1. 의존성 패키지 설치
agents-cli install

# 2. 로컬 개발 플레이그라운드 웹 UI 실행 (기본 포트: 8000)
agents-cli playground

# 3. Uvicorn을 통해 FastAPI 서버 직접 실행
uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000 --reload

# 4. 코드 품질 검사 (Linting)
agents-cli lint

# 5. 단위 및 통합 테스트 실행
uv run pytest tests/unit tests/integration

# 6. 에이전트 평가(Eval) 파이프라인 실행
agents-cli eval

# 7. Google Cloud Agent Runtime 배포
gcloud config set project <your-project-id>
agents-cli deploy

# 8. 배포된 에이전트를 Gemini에 등록
agents-cli publish gemini
```

4. 🛠️ 프로젝트 관리 명령어

명령어	기능
`agents-cli scaffold enhance`	CI/CD 파이프라인과 Terraform 인프라 코드 추가
`agents-cli infra cicd`	CI/CD 파이프라인 및 인프라 전체를 단일 명령어로 원클릭 설정
`agents-cli scaffold upgrade`	사용자 커스터마이즈 코드를 유지하면서 템플릿을 최신 버전으로 자동 업그레이드

5. 요청 처리 흐름: uvicorn에서 Agent까지

`app/fast_api_app.py` 를 기준으로, 클라이언트가 보낸 HTTP 요청이 `root_agent` 까지 도달하는 경로입니다.

1) 서버 기동

```
uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000
```

- `load_dotenv()` 로 환경 변수를 로드합니다.
- `get_fast_api_app(...)` (ADK 제공)으로 FastAPI 앱을 생성하여 기본 웹 UI 및 API 라우트를 등록합니다.
- `shared://session` , `shared://artifact` URI를 통해 서비스 인스턴스를 공유합니다.
- `attach_reasoning_engine_routes(app)` 로 `/api/reasoning_engine` 라우트를 등록합니다.

2) Lifespan: 시작 시 1회 실행

- `app/agent.py` 에서 `adk_app` , `root_agent` 를 로드합니다.
- `Runner(app=adk_app, session_service=..., artifact_service=...)` 를 생성해 `app.state.runner` 에 보관합니다.
- `attach_a2a_routes(...)` 로 A2A 라우트를 `/a2a/app` 에 연결합니다.

3) 요청별 라우팅

들어오는 요청	등록 주체	처리 함수 / 역할
`POST /run_sse`	ADK (`api_server.py`)	`agents-cli playground` SE 스트리밍 엔드포인트
`POST /run`	ADK	전체 이벤트를 모아 단일 JSON 배열로 일괄 반환
`POST /a2a/app` (JSON-RPC)	`app_utils/a2a.py`	`A2aAgentExecutor(runner=...)` 가 요청을 받아 Runner 실행
`POST /api/reasoning_engine`	`app_utils/reasoning_engine_adapter.py`	Cloud 콘솔 플레이그라운드용 인터페이스

4) `/run_sse` 내부 실행 흐름

```
HTTP POST /run_sse (JSON 요청)
  |
  ▼
api_server.run_agent_sse()
  |  get_runner_async("app")
  |     └ 캐시된 Runner 인스턴스 획득
  ▼
Runner.run_async(user_id, session_id, new_message)
  |  1) session_service에서 세션 조회 (없으면 자동 생성)
  |  2) new_message를 세션 이벤트로 기록
  |  3) plugin 훅 실행 후 root_agent 구동
  ▼
root_agent (LlmAgent "my_agents_cli")
  |  프롬프트 + 등록 도구(get_weather, get_current_time)로 Gemini 호출
  |  ├─ 도구 호출 필요 시 → function_call 발생 및 로컬 함수 실행 후 결과 피드백 (반복)
  |  └─ 최종 텍스트 생성 완료 시 → 응답 반환
  ▼
각 Event를 SSE("data: {...}\n\n") 청크 스트림으로 클라이언트에 실시간 전송
```

---

6. 관측 가능성 (Observability) 및 A2A 지원

- **관측 가능성**: Cloud Trace, BigQuery, Cloud Logging으로 텔레메트리 데이터가 자동 전송되도록 내장되어 있습니다.
- **A2A Inspector**: 본 에이전트는 A2A 프로토콜을 완벽히 지원하며, A2A Inspector로 상호 운용성을 검증할 수 있습니다.